

Table of Content

1. Introduction	1
1.1 Background	1
1.2 Problem Definition	1
1.3 Proposed Solution	1
2. Objective	
2.1 General Objective	2
2.2 Specific Objective	2
2.3 Prototype Scope	2
3. System Analysis	
3.1 Functional Requirements	3
3.2 Non Functional Requirements	3
4. System Design	
4.1 Architecture Design	4
4.2 Use Case Diagram	4
4.3 Data Flow Diagrams	5
4.4 Class Diagram	7
4.5 Sequence Diagram	8
4.6 Physical Entity Relationship Diagram	9
4.7 Api Design	9
5. Implementation	10
5.1 Backend	10
5.2 Frontend	11
6. Testing	12
7. Limitation	13
8. Future Enhancements	14
9. Conclusion	15
10. Appendices	16
11. References	17

Online Student Attendance Management System

Chapter 1: Introduction

1.1 Background

The Online Student Attendance Management System (OSAMS) is a web-based application designed to automate and manage student attendance in a classroom environment. It serves as a digital alternative to the traditional pen-and-paper method of recording attendance, aiming to improve efficiency, accuracy, and accessibility.

While replacing manual attendance tracking is not a new concept, advancements in internet technologies and online databases have made such systems more practical and reliable. OSAMS leverages these modern technologies to simplify attendance management, reduce human error, and ensure secure data handling.

1.2 Problem Definition

Although the traditional manual attendance process is functional, it has several limitations. Being paper-based, it is time-consuming, inefficient, and prone to human error. Teachers require a reliable and convenient method to record attendance, while students need access to accurate information about their attendance status and overall record. In the current manual workflow, teachers first record attendance on paper, after which an IT staff member must manually transfer this information into a database. This multi-step process introduces multiple points where mistakes can occur, such as data entry errors, misplaced records, or inconsistencies between physical and digital data.

Therefore, there is a need to reduce these potential sources of error while providing teachers with an efficient recording mechanism and students with a dependable way to view their attendance records. The proposed system aims to address these challenges by automating attendance management and centralizing data handling.

1.3 Proposed Solution

The system provides:

- Authentication for students and teachers
- Teacher interface to mark attendance by subject and date
- Student dashboard to view attendance summary
- Database storage of attendance records

Chapter 2: Objectives

2.1 General Objective

To design and implement a prototype attendance management system following SAD principles.

2.2 Specific Objective

- Analyze attendance tracking requirements
- Design database schema and system architecture
- Implement REST API backend
- Create a simple frontend interface
- Demonstrate marking and viewing attendance

2.3 Prototype Scope

The system represents a functional prototype intended to:

- Demonstrate requirement analysis
- Validate database and architectural design
- Showcase interaction between teacher and student roles
- Illustrate REST-based system communication

It is not intended as a full institutional deployment system and omits enterprise-level concerns such as security hardening, load handling, and distributed scaling.

Chapter 3: System Analysis

3.1 Functional Requirements

The system must provide the following functional capabilities:

- **User Authentication**
 - Login functionality for students and teachers
 - Administrator access may be supported but is not required for the prototype
- **Teacher Requirements**
 - Ability to mark student attendance based on date and subject
 - Ability to update attendance records for students in a specific class
- **Student Requirements**
 - Ability to view personal attendance status
 - Ability to view basic statistics related to attendance (e.g., percentage or summary)
- **Data Management**
 - Attendance data must be stored in a database for efficient querying and retrieval
 - Support for bulk updates to attendance records

3.2 Non-Functional Requirements

- **Performance**
 - The system should provide fast response times for login, attendance marking, and data retrieval operations.
- **Usability**
 - The system should offer a simple and user-friendly interface suitable for both teachers and students.
- **Security**
 - The system should ensure secure user authentication and restrict access to authorized users only.
- **Reliability**
 - The system should maintain reliable data persistence to prevent loss of attendance records.
 - Basic error handling should ensure system stability during normal operation.
- **Scalability**
 - The system should be scalable to support multiple subjects, classes, and increasing numbers of users.
- **Accessibility**
 - The system should be web-based and accessible across common devices and browsers.

Chapter 4: System Design

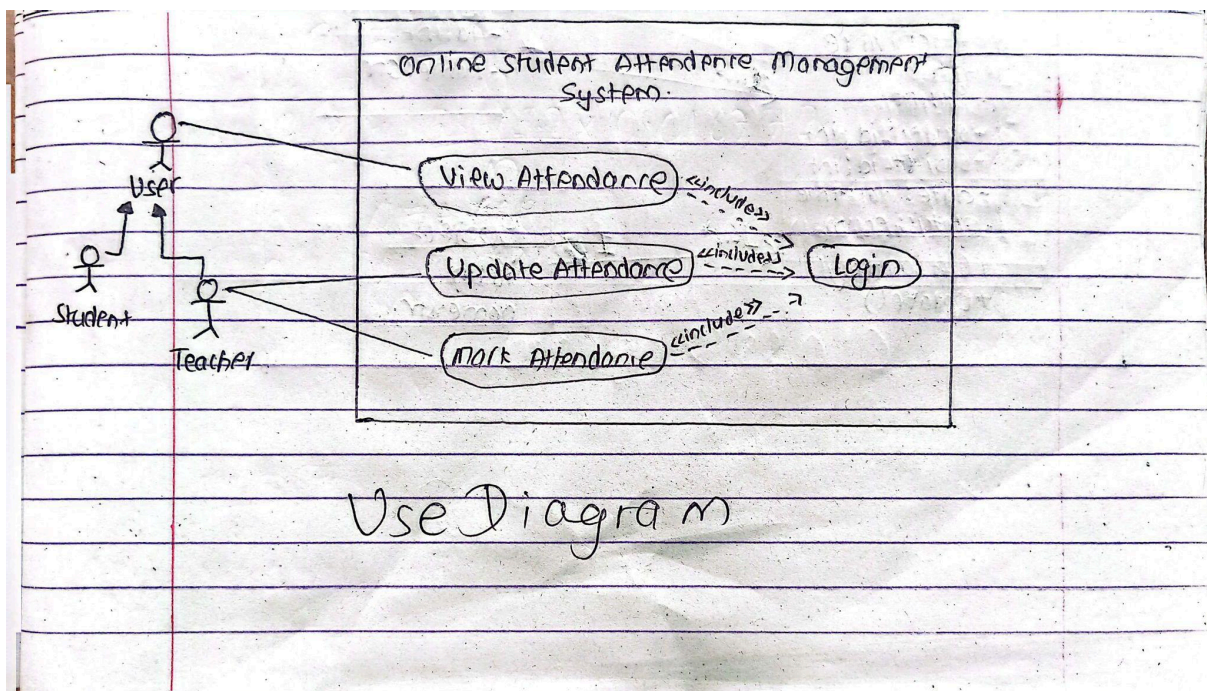
4.1 Architecture Overview

The system follows a **3-tier architecture**:

Tier	Stack
Frontend	HTML, CSS, JavaScript
Backend	FastAPI
Database	SQLite

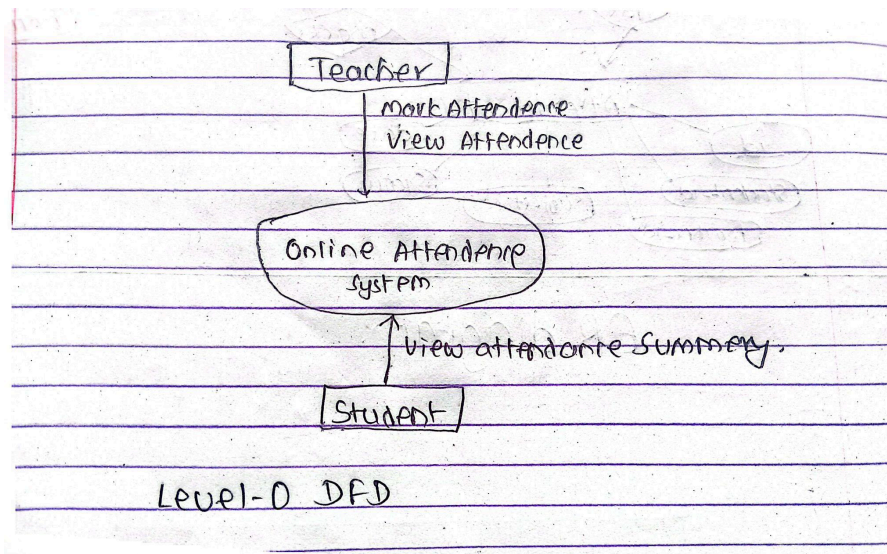
Basic Flow: Browser → FastAPI API → SQLite Database

4.2 Use Case Diagram

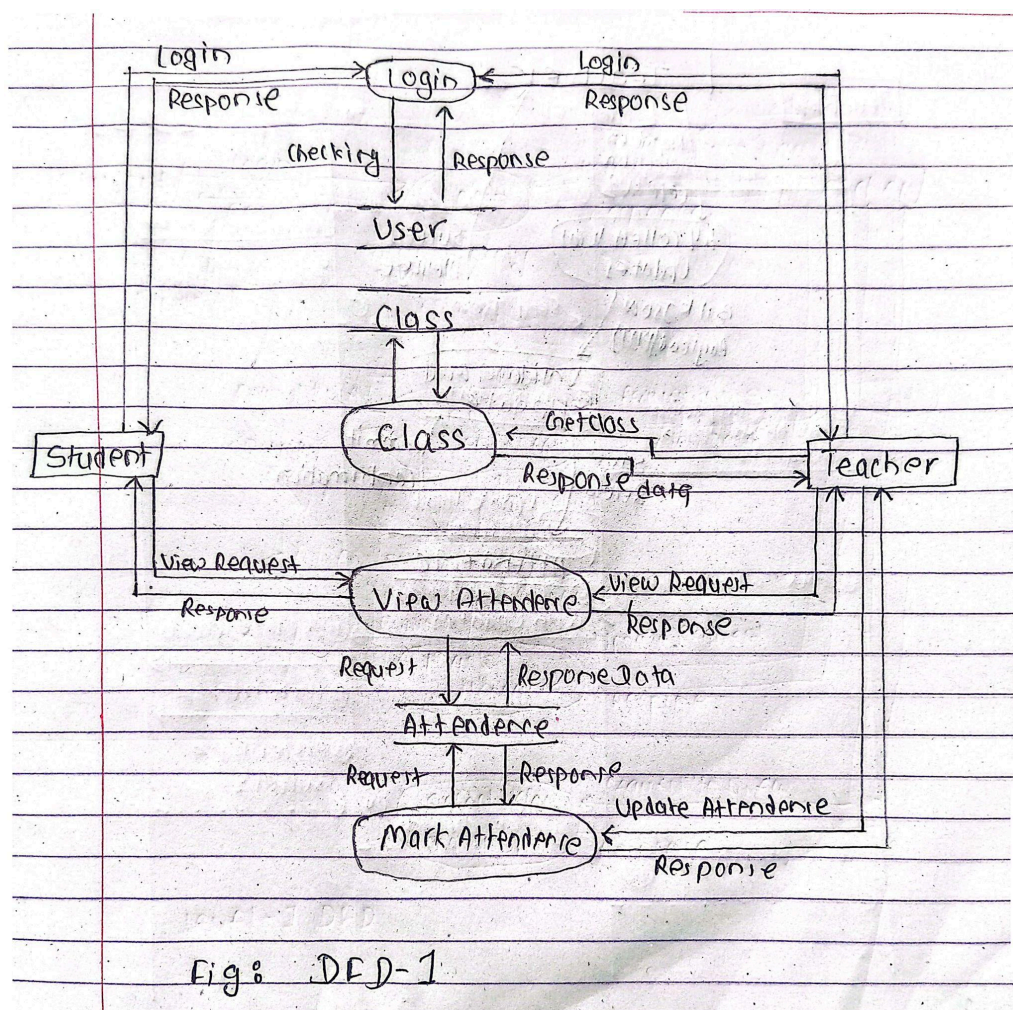


4.3 Data Flow Diagram (DFD)

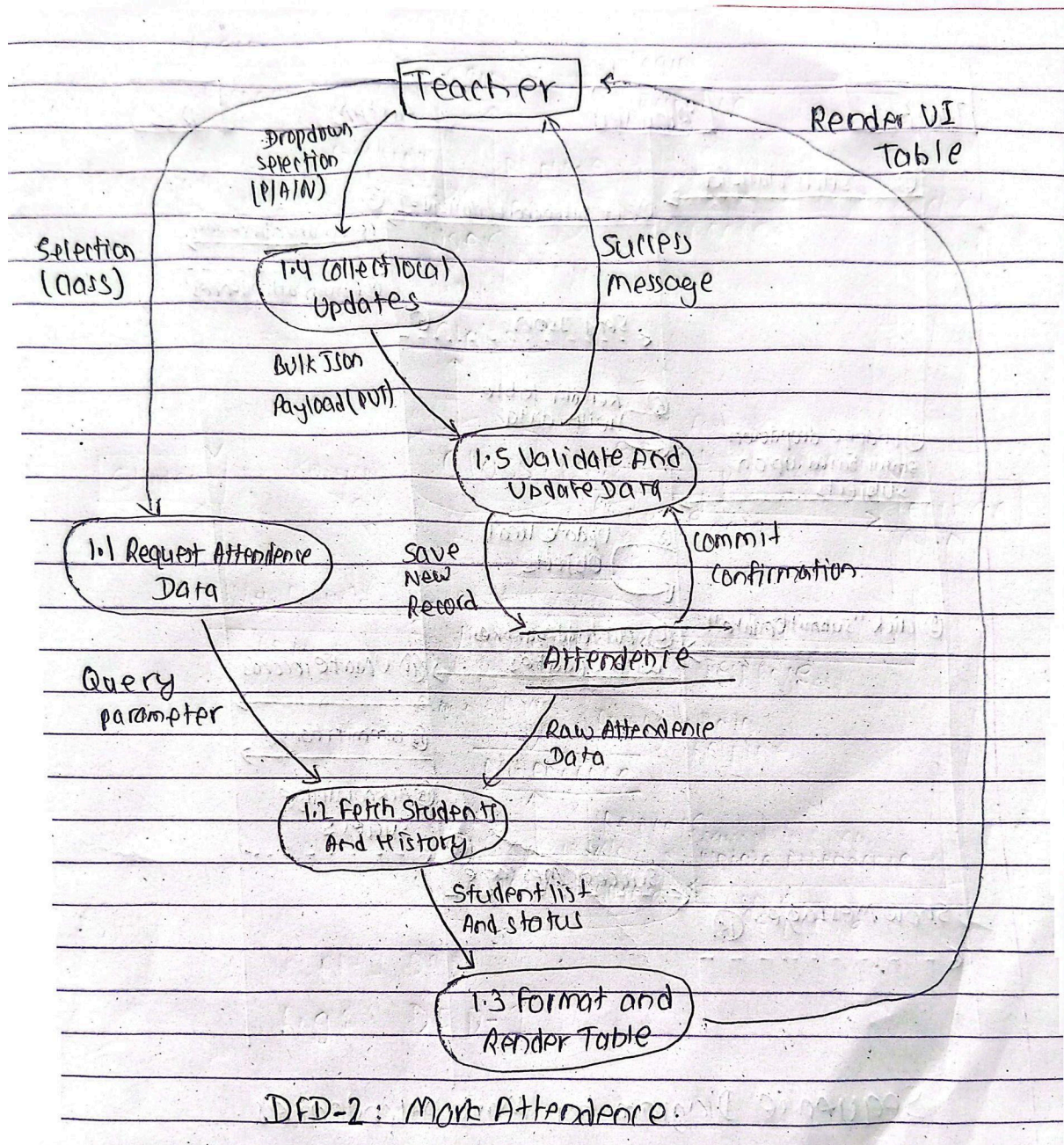
4.3.1 DFD-0



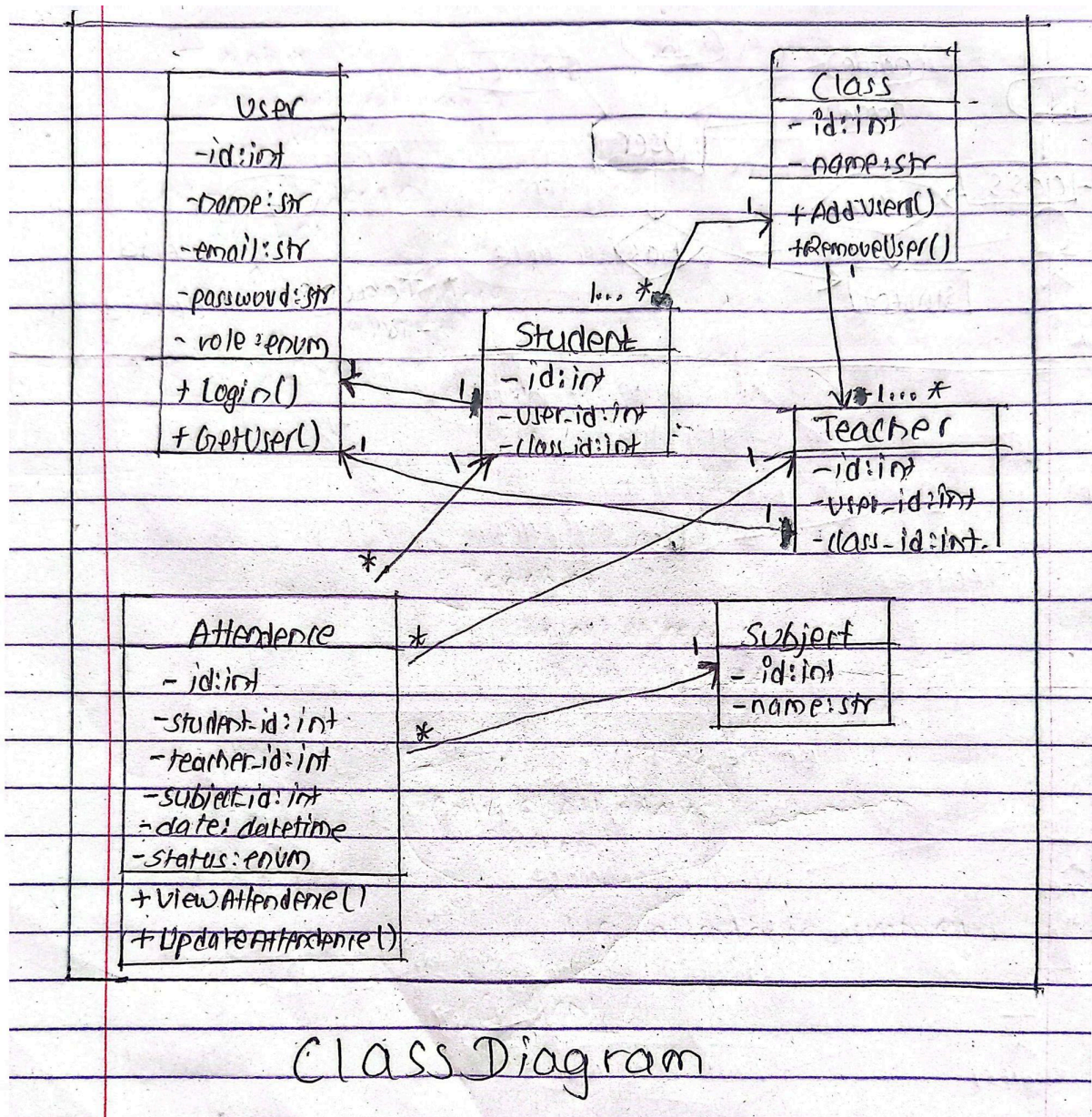
4.3.2 DFD-1



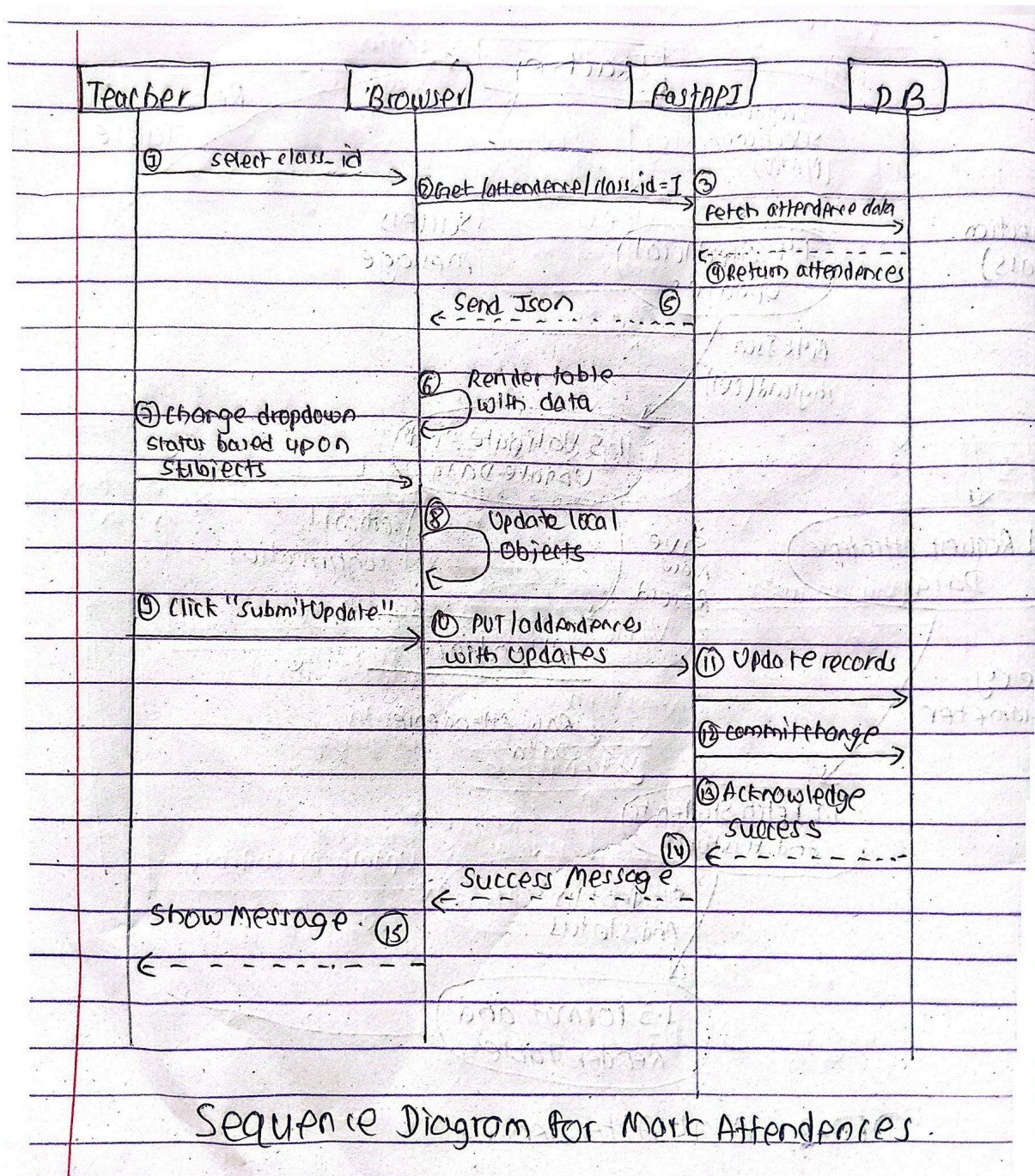
4.3.3 DFD-2(for marking attendance)



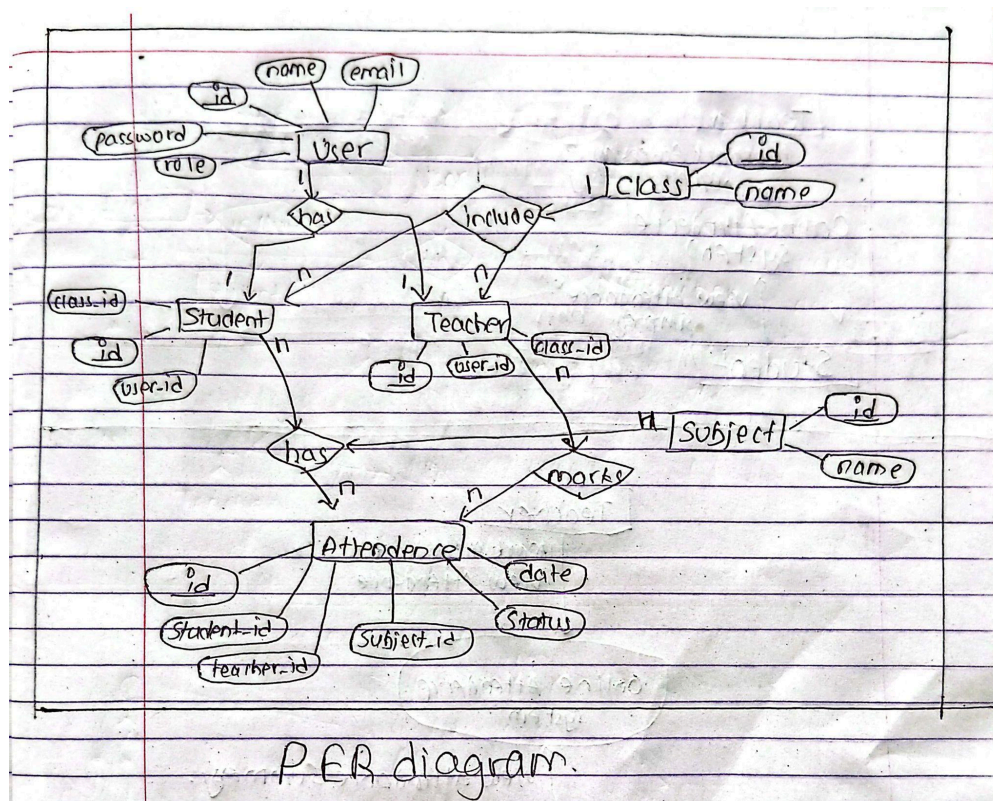
4.4 Class Diagram



4.5 Sequence Diagram (for marking attendance)



4.6 Physical Entity Relationship(PER)Diagram



4.7 Api Design

Endpoint	Method	Input	Output	Purpose
/auth/login	POST	email, password	id, respective_id, name, role	Login and page redirect
/auth/register	POST	name, email, password, class_id, role	id, respective_id, name, role	Register new users
/api/attendance	GET	student_id	attendance_summary	Student view per subject
/api/attendance/teacher	GET	class_id	subjects, students, attendance_data	Teacher view for marking
/api/attendances	PUT	updates_list	success_message	Bulk attendance update

Chapter 5: Implementation

5.1 Backend

The backend of the system is implemented using Python with the FastAPI framework, which provides a fast and modern way to build web APIs. The SQLAlchemy ORM is used for database interactions, simplifying query handling and allowing the system to interact with the database in an object-oriented manner instead of writing raw SQL queries.

For the prototype, an SQLite database is used. This lightweight, file-based database is sufficient for development and testing, eliminating the need for a full-scale database server like PostgreSQL. The database schema is simple, with clearly defined tables to store user information, attendance records, and other necessary data.

The backend exposes RESTful API endpoints for core operations, including user login and registration, fetching user and attendance data, and marking attendance. Additionally, the system supports bulk updates to attendance records, enabling teachers to efficiently update multiple entries at once.

This architecture ensures that the backend is efficient, reliable, and easy to maintain, while providing all the necessary functionality for the prototype system.

Example: Attendance Table

The **Attendance** table stores attendance records for each student, subject, and date. It uses an enumeration to represent attendance status (**PRESENT**, **ABSENT**, or **NOT_SET**) and enforces uniqueness for each student-subject-date combination.

```
class AttendanceEnum(Enum):
    PRESENT = "P"
    ABSENT = "A"
    NOT_SET = "N"

class Attendance(Base):
    __tablename__ = "attendance"
    __table_args__ = (
        UniqueConstraint('student_id', 'subject_id', 'date', name='uq_student_subject_date'),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    student_id: Mapped[int] = mapped_column(Integer, ForeignKey("students.id"))
    subject_id: Mapped[int] = mapped_column(Integer, ForeignKey("subjects.id"))

    date: Mapped[str] = mapped_column(String(10), nullable=False)
    status: Mapped[str] = mapped_column(
        String(1), nullable=False, server_default=sql.expression.text(f'"{AttendanceEnum.NOT_SET.value}"')
    )

# Relationships
```

```
student: Mapped["Student"] = relationship("Student", back_populates="attendance")
subject: Mapped["Subject"] = relationship("Subject", back_populates="attendance")
```

Explanation:

- **AttendanceEnum** defines the allowed attendance statuses.
- **UniqueConstraint** ensures that a student can have only one attendance entry per subject per date.
- **Mapped columns** define the table fields with types and constraints.
- **Relationships** link attendance records to the corresponding Student and Subject objects for easy querying.

5.2 Frontend

The frontend of the system is designed to provide an intuitive and user-friendly interface for both teachers and students. It is web-based, ensuring accessibility across devices and browsers. The interface is organized based on user roles, with separate views for teachers and students.

Teacher Interface

The teacher interface provides all the tools required to manage attendance efficiently:

- **Subject Dropdown:** Allows the teacher to select a subject or class for which attendance is to be recorded.
- **Dynamic Student Table:** Displays a list of students for the selected subject, dynamically populated from the database.
- **Status Selection per Cell:** Teachers can mark each student's attendance status as Present (P), Absent (A), or Not Set (N).

Submit Updates Button: Sends the attendance data to the backend and updates the database. Bulk updates are supported to allow marking multiple records at once.

Student Interface

The student interface provides a read-only view of attendance, giving students easy access to their records:

- **Attendance Table:** Displays personal attendance status for all subjects.
- **Percentage Summary:** Shows overall attendance statistics, such as the percentage of classes attended, allowing students to track their progress.

Login Page

All users must log in before accessing the system:

- **Email/Password Input:** Users enter their credentials to authenticate.
- **Role-Based Redirect:** After login, users are redirected to their respective dashboards based on their role (teacher or student).

The frontend is designed with simplicity in mind, ensuring that both teachers and students can interact with the system efficiently without extensive training. The combination of dynamic tables, dropdowns, and role-based dashboards ensures usability, clarity, and quick access to critical attendance information.

Chapter 6: Testing

The system underwent functional testing to ensure that all core features operate correctly and reliably. The following aspects were tested:

- **User Login**
 - Verified that both teacher and student accounts can log in successfully.
 - Checked that invalid credentials are handled properly, preventing unauthorized access.
- **Attendance Marking**
 - Confirmed that teachers can mark attendance for students using the dynamic table.
 - Ensured that the selection of Present (P), Absent (A), or Not Set (N) is recorded correctly.
- **Database Updates**
 - Verified that attendance records are properly stored and updated in the database.
 - Tested bulk update functionality to ensure multiple records can be modified simultaneously without errors.
- **Student Dashboard**
 - Checked that students can view their attendance summary accurately.
 - Confirmed that the percentage summary reflects the correct attendance calculations.
- **Data Integrity**
 - Ensured that all attendance statuses (P/A/N) are displayed correctly and match the backend records.
 - Tested that duplicate or conflicting entries are prevented by database constraints.

This testing phase confirms that the system meets the functional requirements and that teachers and students can interact with the system efficiently and reliably.

Limitations

While the prototype demonstrates the core functionality of the Online Student Attendance Management System (OSAMS), it has several limitations:

- **Password Security:** The prototype currently lacks cryptographic hashing for user credentials, storing passwords in plain text for demonstration purposes.
- **Authentication Tokens:** The system does not implement authentication tokens or session management for secure and persistent logins.
- **Single-Class Demonstration:** The prototype currently supports only a single-class environment for demonstration purposes.
- **Database Limitations:** SQLite is used for simplicity, but it is not suitable for large-scale, production-ready deployments.
- **User Interface:** The frontend has basic styling and lacks advanced design or responsiveness.

These limitations highlight areas for future improvement, such as implementing secure authentication, using a production-ready database, expanding multi-class support, and enhancing the user interface.

Future Enhancements

The prototype system provides a solid foundation for attendance management, but several improvements can be implemented in the future to enhance functionality, security, and usability:

- **JWT Authentication:** Implement JSON Web Tokens for secure and persistent user sessions.
- **Password Encryption:** Store passwords securely using hashing algorithms to protect user credentials.
- **Multi-Class Support:** Extend the system to handle multiple classes, subjects, and sections efficiently.
- **Analytics Dashboard:** Provide teachers and administrators with visual insights into attendance trends and statistics.
- **Mobile-Responsive UI:** Enhance the frontend to be fully responsive, ensuring usability across desktops, tablets, and mobile devices.
- **Email/SMS Notifications:** Introduce automated notifications to alert students about attendance updates or reminders.

These enhancements will make the system more secure, scalable, and user-friendly, bringing it closer to a production-ready solution.

Conclusion

This prototype system successfully demonstrates the practical application of System Analysis and Design (SAD) principles in developing an online attendance management solution.

- **Analysis:** The inefficiencies and limitations of the manual attendance process were identified, highlighting the need for automation.
- **Design:** A clear database schema was created, along with RESTful API endpoints and a corresponding frontend interface to meet user requirements.
- **Implementation:** A functional backend using FastAPI and a simple web-based frontend were developed, providing core features such as login, attendance marking, and summary views.

The system is scalable, user-friendly, and efficient, serving as a demonstration prototype for classroom attendance management. While it has certain limitations in security, UI, and multi-class support, it provides a solid foundation for future enhancements and a practical example of SAD principles applied in a real-world scenario.

Screenshots / Appendices

- **Figure 1: Unified Login Interface**

Login

Email

Password

Login

teacher@test.com
student@test.com
Password: Test1234!

- **Figure 2: Teacher Attendance Management Dashboard**

Class Attendance

Logout

Select Subject:

Mathematics

Student

Mathematics

Physics

Chemistry

Biology

Computer Science

Submit Update

	2024-06-01	2024-06-02	2024-06-03	2024-06-04	2024-06-05	2024-06-06	2024-06-07
	A	P	P	P	P	P	P

- **Figure 3: Student Attendance Summary View**

Student Attendance

Logout

Total Present: 26 | Total Absent: 9 | Percentage Present: 74.29%

Subject / Date	2024-06-01	2024-06-02	2024-06-03	2024-06-04	2024-06-05	2024-06-06	2024-06-07
Mathematics	A	P	P	P	P	P	P
Physics	P	P	A	P	P	P	P
Chemistry	P	A	P	P	P	A	A
Biology	P	P	P	P	P	A	P
Computer Science	P	A	P	P	A	A	P

References

- KEC Publication and Distribution Pvt. Ltd. (2079 BS). *System Analysis and Design Curriculum Book (BCA/CSIT)*. Kathmandu, Nepal.
- FastAPI Documentation. (2023). *FastAPI – Modern, Fast (high-performance) web framework for building APIs with Python*. Retrieved from: <https://fastapi.tiangolo.com/>
- SQLAlchemy Documentation. (2023). *SQLAlchemy – The Database Toolkit for Python*. Retrieved from: <https://www.sqlalchemy.org/>
- SQLite Documentation. (2023). Retrieved from: <https://www.sqlite.org/>